

Technical Manual: Densidade Vertical – Musical Density Analysis

This document is a comprehensive, pedagogical technical manual for the core calculation pipeline of the Densidade Vertical application. It bridges high-level design and low-level mathematical implementation.

Math formatting: All formulas use **LaTeX** — inline math in `$...$`, display math on **separate lines** as `$$ ... $$` (StackEdit, Stack Exchange MathJax, KaTeX, GitHub, VS Code Markdown Math). Use `\cdot`, `\times`, or `\log_{10}(1+x)`; avoid bare Unicode operators inside expressions.

1. Overview

1.1 High-level purpose

Densidade Vertical computes a set of **vertical density** metrics for a given vertical slice of music (a chord or simultaneity). The system:

- Takes as input: **notes** (pitches), **dynamics**, **instruments**, and **number of instruments** per note.
- Produces: **interval density** (how “compact” or “dissonant” the pitch set is), **instrument density** (spectral contribution of the chosen instruments and dynamics), a **weighted combination** of both, **spectral moments** (centroid, spread, skewness, kurtosis, flatness, entropy), **texture** and **timbre** descriptors, and an optional **combination tones** layer.
- Optionally integrates **psychoacoustic corrections** (critical-band masking, roughness, loudness) and a **calibrated decay parameter** λ for interval density.

The pipeline is **deterministic** given the same input and configuration; it does not perform audio signal processing—it works on symbolic note lists (e.g. from manual entry, XML, or MIDI).

1.2 Design principles

- **Single entry point:** All metrics for one vertical slice are computed by one function, `calculate_metrics(input_data)` (in `data_processor`).
- **Layered metrics:** Raw densities (interval, instrument) → normalised weighted density → refinement by spectral spread → final total density with cohesion, complexity, harmonic ratio, and dynamic boost.
- **Extensibility:** Instrument models are plug-in modules; new instruments can be added gradually without changing the core formulas.

2. Code architecture

2.1 Main entry and data flow

Component	Role
<code>data_processor.calculate_metrics(input_data)</code>	Main entry. Validates input, runs all sub-calculations, assembles and returns <code>(resultados, densidades_instrumento, pitches)</code> .
<code>_validate_and_extract_input(input_data)</code>	Ensures required keys exist and list lengths match; returns <code>(notes, dynamics, instruments, num_instruments, weight_factor)</code> .
<code>_normalize_notes_to_sustenido(notas)</code>	Converts note strings to a canonical sharp form (e.g. flats → sharps) preserving cents.
<code>_assemble_results(...)</code>	Builds the final <code>resultados</code> dictionary from all computed values.

2.2 Core calculation modules

Module	Main functions	Purpose
densidade_intervalar	calculate_interval_density , calculate_interval_density_psychoacoustic , modified_exponential_decay , calibrate_lambda	Interval density (pairwise exponential decay); optional psychoacoustic weighting; lambda calibration.
data_processor	calcular_densidade_ponderada_normalizada , calcular_massa_sonora , instrument loop	Weighted density (min-max / Stevens), sonic mass, per-note instrument density.
spectral_analysis	calculate_spectral_moments , calculate_extended_spectral_moments , calculate_chroma_vector , calculate_harmonic_ratio	Centroid, spread, skewness, kurtosis, flatness, roll-off, entropy; chroma; harmonic ratio (extended moments feed the total-density complexity factor).
timbre_texture_analysis	calculate_texture_density , calculate_timbre_blend , calculate_orchestration_balance	Texture (polyphony, variability, contrast); timbre blend; orchestration balance.
combination_tones	compute_resultant_tones , Tone	Resultant tones (difference tones, cubic) and their integration into the spectral layer.

2.3 Instrument layer

- Each **instrument** is a module under `instrumentos/` exposing:
 - `calcular_densidade(nota, dinamica) → float`
 - `predict_intermediate_dynamics(pitches, pp_values, mf_values, ff_values) → dict of arrays`
- The first instrument in the user's list is used for all notes in the current design; the system can be extended to support per-note instruments. Unknown dynamics are mapped to a default (e.g. `mf`) before calling the instrument.

2.4 Configuration and constants

- **config.py**: `MAX_DENS_GLOBAL` , `USE_LOG_COMPRESSION` , `SPECTRAL_FREQ_MIN` , `SPECTRAL_FREQ_MAX` , `DYNAMIC_LEVELS` , `DEFAULT_LAMBDA` , etc.
- **densidade_intervalar** : Calibrated λ stored in `config/density_params.json` , loaded by `load_calibrated_parameters()` .

3. Mathematical foundations

All formulas below are **code-verified**: they match the implementation in `microtonal` , `densidade_intervalar` , `data_processor` , `spectral_analysis` , and `combination_tones` . Constants (e.g. `A4` , λ bounds, a_2 , a_3) and function names correspond to the codebase. Conventions: MIDI $m \in \mathbb{R}$, semitones, normalised or relative amplitudes as stated.

3.1 Pitch and frequency

Reference (ISO 16): Concert pitch `A4` = 440 Hz; MIDI note number for `A4` is 69. In code: `A4_FREQ` = 440.0 , `A4_MIDI` = 69 (`microtonal.py`).

- **MIDI to frequency**:

$$f = f_{A4} \cdot 2^{(m-69)/12} = 440 \cdot 2^{(m-69)/12}, \quad m \in \mathbb{R}.$$

- **Frequency to MIDI**:

$$m = 69 + 12 \log_2(f/440), \quad f > 0.$$

- **Interval between two notes (semitones):**

$$\Delta_{\text{st}}(i, j) = |m_i - m_j|.$$

- **Spectral spread (pitch range in semitones):**

$$A_{\text{st}} = \max_i m_i - \min_i m_i \quad (\text{over the pitch set}).$$

Used as denominator in refined density when $A_{\text{st}} > 0$.

3.2 Interval density

- **Decay function (microtonal distance δ ; unison = maximum):**

$$\phi(\delta; \lambda) = \begin{cases} 1 & \text{if } \delta = 0, \\ e^{-\lambda\delta} & \text{if } \delta > 0. \end{cases}$$

Implemented in `modified_exponential_decay(delta, lamb)`. Parameter λ is loaded from calibration or defaults to `DEFAULT_LAMBDA` (0.05).

- **Microtonal scale:** In code, interval in semitons Δ_{st} is converted to microtonal steps as $\delta = 2 \cdot \Delta_{\text{st}}$ (24 steps per octave). So $\delta(i, j) = 2|m_i - m_j|$.

- **Raw interval density (sum over all unordered pairs):**

$$D_{\text{int}}^{\text{raw}} = \sum_{i < j} \phi(\delta(i, j); \lambda).$$

- **Normalisation (average per pair):**

$$D_{\text{int}}^{\text{norm}} = \frac{2 D_{\text{int}}^{\text{raw}}}{n(n-1)} \quad (n \geq 2).$$

- **Optional log compression** (when `USE_LOG_COMPRESSION` is True):

$$\tilde{D}_{\text{int}} = \log_{10}(1 + D_{\text{int}}^{\text{norm}}).$$

- **Psychoacoustic path** (`calculate_interval_density_psychoacoustic`): After the above, if psychoacoustic corrections are enabled: apply critical-band masking to unit amplitudes, compute roughness vector and loudness correction; then

- `masking_gain = mean(amps_masked)`,
- `rough_gain = 1 + sum(rough_vec)/sqrt(n)`,
- `loud_gain = mean(loudness_corrected)`,
- $D_{\text{int}}^{\text{final}} = D_{\text{int}}^{\text{scalar}} \cdot \text{masking_gain} \cdot \text{rough_gain} \cdot \text{loud_gain}$,
and optionally apply $\log_{10}(1 + x)$ again (x = scalar density before that step). Per-pair perceptual weighting (register/interval size) is optional and multiplies each ϕ before summing.

3.3 Instrument density

- **Per-note contribution:** For each note i , the instrument module returns a density d_i (depending on note and dynamics). The code then scales by the square root of the number of instruments:

$$d'_i = d_i \cdot \sqrt{n_{\text{instr}, i}}.$$

- **Total instrument density:**

$$D_{\text{inst}} = \sum_i d'_i.$$

3.4 Weighted density (min-max and Stevens)

Implemented in `calcular_densidade_ponderada_normalizada(DI, DV, ...)` with DI = instrument density, DV = interval density.

- **Min-max normalisation** (method "min-max" , configurable maxima):

$$\widehat{D}_{\text{inst}} = \frac{D_{\text{inst}}}{D_{\text{inst,max}}}, \quad \widehat{D}_{\text{int}} = \frac{D_{\text{int}}}{D_{\text{int,max}}}.$$

Defaults: $D_{\text{inst,max}} = 100$, $D_{\text{int,max}} = 10$ (parameters DI_max , DV_max).

- **Alternative: z-score normalisation** (method "z-score"): $\widehat{D}_{\text{inst}} = (D_{\text{inst}} - \mu_{\text{inst}})/\sigma_{\text{inst}}$, $\widehat{D}_{\text{int}} = (D_{\text{int}} - \mu_{\text{int}})/\sigma_{\text{int}}$ with configurable μ , σ (example values in code: $\mu_{\text{inst}} = 50$, $\sigma_{\text{inst}} = 25$; $\mu_{\text{int}} = 5$, $\sigma_{\text{int}} = 2.5$).
- **Optional Stevens power law** (when use_stevens is True; exponents α , β):

$$\widetilde{D}_{\text{inst}} = \widehat{D}_{\text{inst}}^{\alpha}, \quad \widetilde{D}_{\text{int}} = \widehat{D}_{\text{int}}^{\beta}.$$

Defaults: $\alpha = 0.7$, $\beta = 0.4$. If not using Stevens, $\widetilde{D} = \widehat{D}$.

- **Weighted combination** (weight $w \in [0, 1]$, weight_factor in input):

$$D_{\text{pond}} = 10 \cdot (w \widetilde{D}_{\text{inst}} + (1 - w) \widetilde{D}_{\text{int}}).$$

So $w = 0$ uses only interval density, $w = 1$ only instrument density.

3.5 Refined density

- **Refined density** (in code: densidade_refinada_val):

$$D_{\text{ref}} = \begin{cases} \frac{D_{\text{pond}}}{A_{\text{st}}} & \text{if } A_{\text{st}} > 0 \\ D_{\text{pond}} & \text{if } A_{\text{st}} = 0 \end{cases}$$

So when the pitch set has non-zero spread (in semitons), refined density is weighted density divided by that spread; otherwise it equals weighted density (no division by zero).

3.6 Sonic mass and dynamic boost

Implemented in `calcular_massa_sonora` ; result used as `dynamic_boost = sqrt(M)` .

- **Dynamic factors** (exact mapping in code, `fatores_dinamica`):

Dynamic	f_{dyn}
pppp	0.2
ppp	0.3
pp	0.4
p	0.6
mf	1.0
f	1.5
ff	2.0
fff	2.5
ffff	3.0

Unknown dynamics default to 1.0.

- **Sonic mass** (with $d'_i = d_i \sqrt{n_{\text{instr},i}}$ from §3.3):

$$M = \sum_i d'_i \cdot f_{\text{dyn}}(i) \cdot n_{\text{instr},i}.$$

- **Dynamic boost:**

$$\text{boost} = \sqrt{M}.$$

3.7 Spectral moments (centroid, spread, skewness, kurtosis)

Implemented in `calculate_spectral_moments` and `calculate_extended_spectral_moments`. Weights a_i = amplitudes (e.g. densities); $S = \sum_i a_i$. Non-finite values are masked out in code.

- **Centroid (MIDI):**

$$\mu_{\text{MIDI}} = \frac{1}{S} \sum_i a_i m_i.$$

- **Spread (standard deviation in MIDI):**

$$\sigma_{\text{MIDI}} = \sqrt{\frac{1}{S} \sum_i a_i (m_i - \mu_{\text{MIDI}})^2}, \quad \text{with } \sigma_{\text{MIDI}} \geq 0 \text{ (max with 0 in code)}.$$

- **Skewness:**

$$\gamma_1 = \frac{\frac{1}{S} \sum_i a_i (m_i - \mu_{\text{MIDI}})^3}{\sigma_{\text{MIDI}}^3}, \quad \text{with } \gamma_1 = 0 \text{ if } \sigma_{\text{MIDI}} = 0.$$

- **Kurtosis (excess):**

$$\gamma_2 = \frac{\frac{1}{S} \sum_i a_i (m_i - \mu_{\text{MIDI}})^4}{\sigma_{\text{MIDI}}^4} - 3, \quad \text{with } \gamma_2 = 0 \text{ if } \sigma_{\text{MIDI}} = 0.$$

- **Centroid and spread in Hz:** With $f(m) = 440 \cdot 2^{(m-69)/12}$:

$$f_{\text{centroid}} = f(\mu_{\text{MIDI}}) = 440 \cdot 2^{(\mu_{\text{MIDI}}-69)/12},$$

$$\sigma_f = f(\mu_{\text{MIDI}} + \sigma_{\text{MIDI}}) - f(\mu_{\text{MIDI}}) = f_{\text{centroid}} \cdot (2^{\sigma_{\text{MIDI}}/12} - 1).$$

The second equality is exact (code: `midi_to_frequency(centroid_midi + spread_midi) - centroid_freq`).

3.8 Spectral flatness, roll-off, entropy

- **Flatness** (ratio of geometric to arithmetic mean; only $a_i > 10^{-10}$ used to avoid $\log(0)$):

$$\text{flatness} = \frac{\exp\left(\frac{1}{n'} \sum_i \ln a_i\right)}{\frac{1}{n'} \sum_i a_i}, \quad \text{over } i \text{ with } a_i > 0.$$

In code: `np.exp(np.log(nz_amps).mean()) / nz_amps.mean()`.

- **Roll-off (85%):** Amplitudes are cumsummed in pitch order; the roll-off index is the smallest k such that $\sum_{i \leq k} a_i \geq 0.85 S$. The roll-off frequency is $f(m_k)$ where m_k is the MIDI pitch at that index.

- **Entropy** (with $p_i = a_i/S$, only $p_i > 10^{-10}$ to avoid $\log 0$):

$$H = - \sum_i p_i \log_2 p_i \quad (\text{bits}).$$

3.9 Chroma vector

- **Chroma classes:** $c \in \{0, \dots, 11\}$ (C, C#, ..., B). For each pitch m_i , class $c_i = \text{round}(m_i) \bmod 12$ (code: `int(round(p)) % 12`).
- **Chroma energy and normalisation:**

$$E_c = \sum_{i: c_i=c} a_i, \quad \tilde{E}_c = \frac{E_c}{\sum_{c'=0}^{11} E_{c'}} \quad \text{if } \sum_{c'} E_{c'} > 0.$$

The chroma vector is $(\tilde{E}_0, \dots, \tilde{E}_{11})$; normalised so it sums to 1 when there is energy.

3.10 Harmonic ratio

- **Definition:** Ratio of energy in “harmonic” bins (pitches within ± 0.25 semitons of a multiple of 12 semitons above the fundamental) to total energy. Fundamental is $m_{\min}$ if not provided.

$$\text{harmonicRatio} = \frac{\sum_{i \in \mathcal{H}} a_i}{\sum_i a_i}, \quad \mathcal{H} = \{i : |(m_i - m_{\min}) \bmod 12| \leq 0.25 \text{ (in code: } \text{isclose}((m_i - m_{\min}) \bmod 12, 0, \text{atol} = 0.25))\}$$

If total energy is 0, the ratio is 0.

3.11 Cohesion and complexity factors

- **Cohesion factor** (in code: `coesao_factor`):

$$C_{\text{coes}} = \frac{10}{1 + A_{\text{st}}}.$$

So compact verticals (small A_{st}) get a higher factor; wide spreads reduce it.

- **Complexity factor** (from spectral entropy H ; code: `1 + np.log1p(entropy)`):

$$C_{\text{comp}} = 1 + \ln(1 + H).$$

3.12 Total density (final formula)

- **Unnormalised total** (code: before division by `MAX_DENS_GLOBAL`):

$$D_{\text{total}}^{\text{raw}} = D_{\text{ref}} \cdot C_{\text{coes}} \cdot C_{\text{comp}} \cdot (1 - 0.15 \cdot \text{harmonicRatio}) \cdot \text{boost}.$$

- **Normalisation:** $D_{\text{total}}^{\text{norm}} = D_{\text{total}}^{\text{raw}} / D_{\text{max}}$ with $D_{\text{max}} = \text{MAX_DENS_GLOBAL}$ (default 20).
- **Optional log compression** (when `USE_LOG_COMPRESSION` is True):

$$D_{\text{total}} = \log_{10}(1 + D_{\text{total}}^{\text{norm}}).$$

Otherwise $D_{\text{total}} = D_{\text{total}}^{\text{norm}}$.

3.13 Absolute density (reference)

- **Tone count:** $N_{\text{tones}} = n + n_{\text{combo}}$ when combination tones are integrated, else $N_{\text{tones}} = n$ (code: `len(notas) + len(combination_tones_pitches)` when present).
- **Absolute density** (code: `np.log1p(total_tones_count)`):

$$D_{\text{abs}} = D_{\text{pond}} \cdot \ln(1 + N_{\text{tones}}).$$

3.14 Texture metrics (summary)

Implemented in `calculate_texture_density(pitches, instruments_counts)`.

- **Average texture density:** $\sum_i n_{\text{instr}, i}$.

- **Texture polyphony:** $\frac{1}{n} \sum_i n_{\text{instr},i}$ (mean instruments per note).
- **Texture variability:** standard deviation of m_i (semitons).
- **Texture contrast:** $\max_i m_i - \min_i m_i$ (semitons).

3.15 Lambda calibration

Implemented in `calibrate_lambda(experimental_data)`. Reference data: CONSONANCE_RATINGS (e.g. Hutchinson & Knopoff, Malmberg, Kameoka & Kuriyagawa).

- **Experimental data:** Dict mapping interval (semitons) to consonance rating in $[-1, 1]$ (e.g. $0 \rightarrow 1.0$, $5 \rightarrow 1.24$, $2 \rightarrow -0.582$).
- **Prediction:** For each interval k , build a two-note chord and compute raw interval density $D_{\text{int}}(k; \lambda)$. Normalise to $[-1, 1]$ using the **maximum experimental rating** $R_{\text{max}} = \max_j \{\text{rating}_j\}$:

$$\text{pred_norm}_k(\lambda) = 2 \cdot \frac{D_{\text{int}}(k; \lambda)}{R_{\text{max}}} - 1.$$

- **Optimisation:** $\lambda^* = \text{argmin}_{\lambda} \sum_k (\text{pred_norm}_k(\lambda) - \text{rating}_k)^2$, with bounds $\lambda \in [0.01, 1]$, method L-BFGS-B. Optimised λ is stored in `config/density_params.json` and loaded by `load_calibrated_parameters()`.

3.16 Combination tones

Implemented in `combination_tones.compute_resultant_tones`. Model: non-linear distortion $y(t) = x + a_2 x^2 + a_3 x^3$; coefficients in code: $a_2 = 0.02$ (A2_COEF), $a_3 = 0.002$ (A3_COEF). Input primaries: frequencies $f_1 \leq f_2$ (Hz), levels in dB; linear amplitudes A_1, A_2 from dB via $A = 10^{(\text{dB}-70)/20}$ (reference 70 dB).

- **Frequencies (Hz):**
 - Quadratic difference: $f_{\text{diff}} = f_2 - f_1$.
 - Cubic: $f_{2f_1-f_2} = 2f_1 - f_2$, $f_{2f_2-f_1} = 2f_2 - f_1$.
 - Sum (optional): $f_{\text{sum}} = f_1 + f_2$.
- **Relative amplitudes (weights):**
 - Difference / sum: $w_{\text{diff}} = w_{\text{sum}} = |a_2| \cdot A_1 \cdot A_2$.
 - $2f_1 - f_2$: $w_{2f_1-f_2} = |a_3| \cdot \frac{3}{4} \cdot A_1^2 \cdot A_2$.
 - $2f_2 - f_1$: $w_{2f_2-f_1} = |a_3| \cdot \frac{3}{4} \cdot A_2^2 \cdot A_1$.

Only resultants with $f \in [f_{\text{min}}, f_{\text{max}}]$ (default 20–20000 Hz) and weight $\geq \text{min_weight}$ are returned. References: Moore (2013), Rossing, *The Science of Sound*, Ch. 8.

4. Practical example

4.1 Sample input

Consider a single chord (vertical slice) with three notes:

```
input_data = {
    "notes": ["C4", "E4", "G4"],
    "dynamics": ["mf", "f", "mf"],
    "instruments": ["flauta", "flauta", "clarinete"],
    "num_instruments": [1, 1, 1],
    "weight_factor": 0.5,
    "use_stevens": True,
    "alpha": 0.7,
    "beta": 0.4,
    "use_psychoacoustic": True,
    "use_perceptual_weighting": False,
```

```

    "calculate_combination_tones": True,
}

```

- **Notes:** C4 (MIDI 60), E4 (64), G4 (67).
- **Spectral spread:** $A_{st} = 67 - 60 = 7$ semitons.
- **Weight:** $w = 0.5$ (equal balance between interval and instrument density).

4.2 Step-by-step (conceptual)

1. **Validation:** Lists have length 3; all required keys present → extract notes, dynamics, instruments, num_instruments, weight_factor.
2. **Note normalisation:** "C4", "E4", "G4" already in canonical form (e.g. sharp); no change.
3. **Interval density:** Pairs (C4,E4), (C4,G4), (E4,G4) → intervals 4, 7, 3 semitons. With $\lambda \approx 0.05$ and microtonal $\delta = 2 \cdot \Delta_{st}$:

$$\circ \phi(8) = e^{-0.05 \cdot 8}, \phi(14), \phi(6); \text{sum} \rightarrow D_{int}^{raw}. \text{ Optionally normalise and apply log.}$$

4. **Instrument density:** For each note, the instrument module returns d_i for the given dynamics; $d'_i = d_i \cdot \sqrt{1} = d_i$. $D_{inst} = d_1 + d_2 + d_3$.
5. **Weighted density:** Normalise D_{inst} and D_{int} (min-max), then apply Stevens with $\alpha = 0.7, \beta = 0.4$ (so $\tilde{D}_{inst} = \tilde{D}_{inst}^{0.7}, \tilde{D}_{int} = \tilde{D}_{int}^{0.4}$), then:

$$D_{pond} = 10 \cdot (0.5 \cdot \tilde{D}_{inst} + 0.5 \cdot \tilde{D}_{int}).$$

6. **Refined density:** $D_{ref} = D_{pond}/7$.
7. **Combination tones:** From the three primary tones, compute difference and cubic resultants; append their MIDI pitches and weights to the pitch and amplitude lists used for spectral metrics.
8. **Spectral moments:** On the combined (primary + combination) pitches and amplitudes, compute centroid (MIDI and Hz), spread, skewness, kurtosis, flatness, roll-off, entropy.
9. **Chroma and harmonic ratio:** Chroma vector from pitch classes; harmonic ratio from energy in harmonic bins vs total.
10. **Cohesion and complexity:** $C_{coes} = 10/(1 + 7); C_{comp} = 1 + \ln(1 + H)$.
11. **Sonic mass:** $M = \sum_i d'_i \cdot f_{dyn}(i) \cdot 1$; boost = $\sqrt{M}$.
12. **Total density:** $D_{total}^{raw} = D_{ref} \cdot C_{coes} \cdot C_{comp} \cdot (1 - 0.15 \cdot \text{harmonicRatio}) \cdot \text{boost}$; then $D_{total}^{norm} = D_{total}^{raw}/\text{MAX_DENS_GLOBAL}$; if `USE_LOG_COMPRESSION`, $D_{total} = \log_{10}(1 + D_{total}^{norm})$, else $D_{total} = D_{total}^{norm}$.
13. **Absolute density:** $D_{abs} = D_{pond} \cdot \ln(1 + N_{tones})$.

4.3 Expected output structure

Calling `resultados, densidades_instr, pitches = calculate_metrics(input_data)` returns:

- **resultados["density"]:**
interval, instrument, weighted, refined, total, sonic_mass, absolute (all numeric).
- **resultados["spectral_moments"]:**
centroid (frequency, note), spread (deviation), spectral_skewness, spectral_kurtosis, spectral_flatness, spectral_rolloff, spectral_entropy.
- **resultados["additional_metrics"]:**
complexity, harmonic_ratio, chroma_vector.
- **resultados["texture"]:**
Texture density, polyphony, variability, contrast.
- **resultados["timbre"]:**
Timbre blend: timbre_diversity, blend_index, density_variance, family_contributions, timbre_balance, timbre_dominance.

- **resultados["orchestration"]** :
Orchestration: register_balance, density_balance, orchestration_evenness, register_distribution, and aliases orchestration_balance, pitch_balance, instrument_balance.
- **resultados["combination_tones"]** (if enabled):
Number of input/resultant tones, list of resultants (frequency, weight, kind), summary.
- **densidades_instr** : Length 3 (one per input note); each entry is the (possibly sqrt-scaled) instrument density for that note.
- **pitches** : Length 3 (MIDI of C4, E4, G4); combination tones are not appended to this list, but they are used internally for spectral and total density.

4.4 Example numerical ranges (orientation only)

- **Interval density**: Typically positive; depends on λ and number/size of intervals. After log, often in a range like $[0, 1]$ for moderate chords.
- **Instrument density**: Positive; order of magnitude depends on instrument modules (e.g. tens).
- **Weighted density**: With min-max and $w = 0.5$, often in $[0, 10]$.
- **Total**: After normalisation and log, often in $[0, 1]$ or similar; exact values depend on all factors above.

These ranges are indicative; the manual does not fix a single “expected” number so that the implementation can evolve (e.g. new instruments or calibration) without contradicting the document.

5. References to code

Concept	Module	Function / constant
MIDI ↔ Hz	microtonal	midi_to_hz, hz_to_midi; A4_FREQ, A4_MIDI
Interval decay	densidade_intervalar	modified_exponential_decay
Raw interval density	densidade_intervalar	calculate_interval_density
Psychoacoustic interval density	densidade_intervalar	calculate_interval_density_psychoacoustic
Weighted density	data_processor	calcular_densidade_ponderada_normalizada
Sonic mass	data_processor	calcular_massa_sonora
Spectral moments	spectral_analysis	calculate_spectral_moments, calculate_extended_spectral_moments
Chroma	spectral_analysis	calculate_chroma_vector
Harmonic ratio	spectral_analysis	calculate_harmonic_ratio
Total density assembly	data_processor	calculate_metrics (steps 8–12)
Lambda calibration	densidade_intervalar	calibrate_lambda, load_calibrated_parameters
Combination tones	combination_tones	compute_resultant_tones, combination_weigh; A2_COEF, A3_COEF
Publication figures	utils.plotting_style	create_professional_figure, enhance_axes, finalize_figure, DISPLAY_DPI, PUBLICATION_DPI

This manual is intended to be stable with respect to the mathematical model; implementation details (e.g. optional psychoacoustic terms, exact defaults) can be read from the source when needed.

6. Visualization and publication figures

6.1 Plotting style (`utils.plotting_style`)

- **Theme:** Built-in style `seaborn-v0.8-whitegrid` or `ggplot` (no external seaborn required).
- **Palette:** `Viridis` and `magma` (`matplotlib.colormaps`); default colour cycle is `viridis`.
- **Typography:** `Mathtext` is enabled so axis labels and legends can use LaTeX-style math (e.g. λ , δ , f_c) without an external LaTeX install.
- **Layout:** Top and right spines are removed; grid is a subtle grey; `finalize_figure(fig)` applies `tight_layout` before save. Use **DISPLAY_DPI (96)** for on-screen and embedded figures so they fit the monitor; use **PUBLICATION_DPI (300)** only when saving to file (e.g. `fig.savefig(..., dpi=PUBLICATION_DPI)`).

6.2 Main plotting modules

Module	Role
<code>plot_metr_espectrais</code>	Spectral metrics bar chart; chroma and spectral distribution.
<code>plot_spectrogram</code>	Spectrogram-like density view (pitch vs density); optional 3D.
<code>calibration</code>	Decay curve, consonance vs λ , experimental vs model comparison.
<code>statistical_validation</code>	Metrics comparison and boxplots.
<code>timbre_texture_analysis</code>	2×2 orchestration view (3D scatter, density profile, heatmap).
<code>scientific_report_generator</code>	Density, spectral, and chroma plots for PDF reports.

All of these use `create_professional_figure`, `enhance_axes`, and (where applicable) `finalize_figure` from `utils.plotting_style` so that labels use the same conventions as this manual (e.g. δ , $w(\delta)$, λ).

6.3 Running and building

Run the application with `python run.py` or the `densidade-vertical` entry point after `pip install -e .` To build a standalone Windows executable, use `python build_exe.py` (see README and PyInstaller documentation).